❑ **Exercice 4** : *Rotation droite d'un tableau*

Dans cet exercice, on ne tient pas compte des éventuels dépassements de capacité lors des additions ou soustractions sur les entiers et on cherche à écrire, sans utiliser de variable intermédiaire, une fonction permettant d'effectuer la rotation droite d'un tableau. Par exemple, le tableau `{2, 6, 4, 1}` devient `{1, 2, 6, 4}` : c'est-à-dire que tous les éléments sont décalés d'un cran vers la droite, sauf le dernier qui devient le premier.

Q28– Écrire une fonction `void echange(int tab[], int i, int j)` qui prend en argument un tableau d'entiers ainsi que deux indices distincts `i` et `j` valides (c'est-à-dire compris entre 0 et la taille du tableau moins 1, avec $i \neq j$) et échange les éléments situés aux indices `i` et `j` *sans utiliser de variable intermédiaire*.

```
void echange(int tab[], int i, int j)
{
    tab[i] = tab[i] + tab[j];
    tab[j] = tab[i] - tab[j];
    tab[i] = tab[i] - tab[j];
}
```

Q29– Écrire une fonction de signature `void rotation(int tab[], int n)` permettant d'effectuer une rotation droite de tous les éléments d'un tableau *sans utiliser de variable intermédiaire*.

```
void rotation(int tab[], int n)
{
    for (int i=1; i<n;i++)
    {
        echange(tab, i, 0);
    }
}
```

Q30– En proposant un invariant de boucle, prouver que la fonction `rotation` est correcte.

Si $n \leqslant 1$, la boucle ne s'exécute pas et le tableau est inchangé, ce qui est le résultat attendu.
On suppose $n \geqslant 2$. On note T_0 le tableau initial avant l'exécution de la boucle.
Montrons que la propriété $\mathcal{I}(i)$ suivante est un invariant de boucle pour $i \in [\![1; n]\!]$ (au début de chaque tour de boucle d'indice i) :

1. tab[0] $= T_0[i-1]$;
2. tab[k] $= T_0[k-1]$ pour tout $k \in [\![1; i-1]\!]$ (éléments déjà décalés) ;
3. tab[k] $= T_0[k]$ pour tout $k \in [\![i; n-1]\!]$ (éléments inchangés).

Cet état du tableau au début de l'itération i se représente ainsi :

0	1	2	…	$i-1$	i	…	$n-1$
$T_0[i-1]$	$T_0[0]$	$T_0[1]$	…	$T_0[i-2]$	$T_0[i]$	…	$T_0[n-1]$
pivot	éléments déjà décalés				éléments inchangés		

• Initialisation ($i = 1$) Avant le premier tour, aucun élément n'a été déplacé : le point (1) donne tab[0] $= T_0[0]$, le point (2) est vérifié par vacuité ($[\![1; 0]\!] = \emptyset$), et le point (3) assure que tab[k] $= T_0[k]$ pour tout $k \in [\![1; n-1]\!]$. L'invariant $\mathcal{I}(1)$ est donc vérifié.
• Conservation Supposons $\mathcal{I}(i)$ vrai pour $1 \leqslant i < n$. L'unique opération du corps de boucle est `echange(tab, i, 0)`, qui permute tab[0] et tab[i] sans modifier les autres cases :

— tab[i] prend l'ancienne valeur de tab[0], soit $T_0[i-1]$;
— tab[0] prend l'ancienne valeur de tab[i], soit $T_0[i]$;
— les cases d'indices 1 à $i-1$ et celles d'indices $i+1$ à $n-1$ restent inchangées.

Le tableau devient donc :

0	1	…	$i-1$	i	$i+1$	…	$n-1$
$T_0[i]$	$T_0[0]$	…	$T_0[i-2]$	$T_0[i-1]$	$T_0[i+1]$	…	$T_0[n-1]$
pivot	éléments déjà décalés				éléments inchangés		

Ce qui correspond exactement à la propriété $\mathcal{I}(i+1)$. L'invariant est conservé.
• En sortie de boucle, $\mathcal{I}(n)$ garantit que :

— tab[0] $= T_0[n-1]$;
— tab[k] $= T_0[k-1]$ pour tout $k \in [\![1; n-1]\!]$.

Le tableau final contient bien la rotation droite de T_0, ce qui prouve la correction de l'algorithme.